

INSERT INTO y SELECT

Teoría

1. Insertar datos en una tabla — INSERT INTO

¿Qué es INSERT INTO?

INSERT INTO es el comando que permite **añadir nuevos registros** (filas) en una tabla de la base de datos. Cada fila insertada representa un nuevo dato almacenado.

Sintaxis general

```
INSERT INTO nombre_tabla (columna1, columna2, columna3, ...)  
VALUES (valor1, valor2, valor3, ...);
```

Explicación de los componentes

- `nombre_tabla`: Nombre de la tabla donde queremos insertar los datos.
- `columna1, columna2, ...`: Lista ordenada de las columnas que vamos a rellenar.
- `valor1, valor2, ...`: Valores que queremos insertar, respetando el tipo de dato definido en cada columna.

Reglas importantes

- Las cadenas de texto (VARCHAR) y las fechas (DATE) deben ir entre comillas simples ' '.
- Debemos insertar valores en las columnas que sean NOT NULL.
- Podemos omitir columnas que tengan valores DEFAULT.
- Respetar siempre el orden de columnas y valores.

Ejemplo práctico

```
INSERT INTO clientes  
  (nombre,  
   apellidos,  
   fecha_registro,  
   telefono,  
   email,  
   cuenta_bancaria)  
VALUES (  
  'Ana',  
  'López García',  
  '2024-04-30',  
  '612345678',  
  'ana.lopez@email.com',  
  'ES12345678901234567890'  
);
```

2. Consultar datos en una tabla — SELECT

¿Qué es SELECT?

SELECT es el comando que utilizamos para **consultar y visualizar** los datos almacenados en una tabla.

Sintaxis general

```
SELECT * FROM nombre_tabla;
```

Explicación de los componentes

- SELECT: Palabra clave para realizar consultas.
- *: Indica que queremos seleccionar **todas las columnas**.
- FROM nombre_tabla: Indica de qué tabla queremos recuperar los datos.

Reglas importantes

- SELECT * muestra **todos los registros y todas las columnas**.
- Podemos especificar columnas concretas:

```
SELECT nombre, apellidos FROM clientes;
```

- Si no hay registros, el resultado será una tabla vacía.

Ejemplo práctico

```
SELECT * FROM clientes;
```

Parte A: Inserción de registros correctos

Objetivo

Insertar correctamente un registro en la tabla clientes respetando los tipos de datos y restricciones.

Instrucciones

1. Ejecuta un INSERT INTO correctamente.
2. Tras el INSERT, realiza un SELECT * FROM clientes; para verificar que los datos se han insertado correctamente.

Comando de ejemplo

```
INSERT INTO clientes
    (nombre,
     apellidos,
     fecha_registro,
     telefono,
     email,
     cuenta_bancaria)
VALUES (
    'Mario',
    'López Fernández',
    '2024-05-02',
    '698765432',
    'mario.lopez@email.com', 'ES09876543210987654321');
```

Verificación

- Comprueba que los datos se han insertado.
- Verifica los valores por defecto (Vigo, Pontevedra).
- Confirma que se ha generado el id automáticamente.

Parte B: Inserciones provocando errores y análisis

a) Error por dejar campo NOT NULL vacío

Acción: Realizar un INSERT INTO omitiendo un campo obligatorio (email).

Observaciones:

- Anotar el mensaje de error recibido.
- Explicar por qué ocurre el error.

b) Error por insertar un email duplicado

Acción: Insertar un registro con un email ya existente.

Observaciones:

- Anotar el mensaje de error recibido.
- Identificar la restricción violada (UNIQUE).

c) Error por tipo de dato incorrecto

Acción: Intentar insertar un valor no válido en fecha_registro.

Observaciones:

- Anotar el mensaje de error recibido.
- Explicar qué tipo de dato esperaba SQL.

Parte C: Verificación final

Comando de comprobación

```
SELECT * FROM clientes;
```

Elementos a verificar

- Confirmar qué registros existen tras los éxitos y errores.
 - Comprobar que la integridad de la tabla no se ha visto comprometida.
-

◇ Reflexión Final

- ¿Qué has aprendido sobre la importancia de validar correctamente los datos antes de insertarlos?
 - ¿Qué errores fueron más frecuentes y cómo podrías evitarlos en el futuro?
-

Recuerda:

- Escribe los comandos de manera precisa.
- Después de cada INSERT, realiza un SELECT para verificar.
- Anota cualquier error y reflexiona sobre su causa.